CTF Writeup

Cleartext Catch — CTF Writeup

Platform: Perfect Root (AfricaHackon) **Room:** Cleartext Catch **Author:** LordSudo **Difficulty:** Easy **Points:** 100 **Category:** Network Forensics / PCAP Analysis **Result:** Solved (1/1 flag captured)

Description

> A packet capture from an internal audit needs review: was any sensitive file transferred across this segment in the clear? You're given the full capture, including unrelated background traffic.

Flag

`r00t{ftp_cr3ds_1n_pl41n_s1ght}`

Vulnerability

FTP transmits both credentials and file contents in cleartext. An on-path observer (or anyone with the capture) can recover the username, password, and the transferred file without any crypto attack. (CWE-319: Cleartext Transmission of Sensitive Information.)

Recon

`capture.pcap` (159 frames). Protocol hierarchy:

```
eth > ip > tcp > http  (18 frames)
            tcp > ftp   (14 frames)
            tcp > ftp-data (1 frame)
          udp > dns     (8 frames)
```

Endpoints showed one internal host (10.20.0.15) talking to an FTP server (10.20.0.5:21). The HTTP traffic (page0.html … x2) is deliberately irrelevant — the "unrelated background traffic" the description warned about.

Step 1 — Protocol overview

```
PCAP='./Cleartext Catch/capture.pcap'
tshark -r "$PCAP" -q -z io,phs
tshark -r "$PCAP" -q -z conv,tcp
tshark -r "$PCAP" -q -z endpoints,ip
```

Key TCP conversation:

```
10.20.0.15:53201 <-> 10.20.0.5:21   (control)
10.20.0.15:53202 <-> 10.20.0.5:51205 (data / PASV)
```

Step 2 — Recover FTP credentials and the transfer

```
tshark -r "$PCAP" -Y "ftp || ftp-data" -T fields \
  -e ip.src -e ip.dst -e ftp.request.command -e ftp.request.arg
```

Output: USER employee13 PASS Harbor_Fal12 TYPE I PASV RETR backup.zip QUIT

Credentials and the filename are visible in the clear.

Step 3 — Carve the transferred file

Find the ftp-data TCP stream number:

```
tshark -r "$PCAP" -Y "ftp-data" -T fields -e tcp.stream
7
```

Follow the stream raw and carve the bytes:

```
tshark -r "$PCAP" -q -z follow,tcp,raw,7 \
  | grep -oE '[0-9a-f]{40,}' \
  | xxd -r -p > backup.zip

file backup.zip
# backup.zip: Zip archive data, ... uncompressed size 105, method=deflate
```

Step 4 — Unlock and read

The zip is password-protected. The password is the FTP password (reused):

```
unzip -P Harbor_Fal12 backup.zip
cat notice.txt
```

Contents: Internal backup - Q3 credentials rotation Do not share outside IT. flag: r00t{ftp_cr3ds_1n_pl41n_s1g

Issues Hit & How They Were Solved

1. **Zsh parse error on `<7>` in my notes.** Cause: I copied a placeholder literally. Angle brackets are zsh redirection. Fix: use the bare stream number `7`.
2. `tshark --export-objects ftp-data,.` did not always yield the file. Cause: `ftp-data` object extraction is less reliable than HTTP/TFTP in older tshark builds. Fix: follow the raw TCP stream and carve the hex with `grep + xxd`.
3. `awk 'NF==2 && $1 ~ /^[0-9a-f]+$/ {print $1}'` **produced an empty file.** Cause: tshark's `follow,tcp,raw` output prefixes each line with an offset AND the hex payload, but the offset column can contain non-hex characters and the line layout differs by tshark version. Fix: `grep -oE '[0-9a-f]{40,}'` — grab contiguous hex runs ≥40 chars, which isolates the payload unambiguously.
4. `http.content_disposition` **field error from tshark.** Cause: field not available in this tshark build. Fix: drop it; use `http.file_data` and `http.content_type` instead.

Real-World Impact — Why This Matters

FTP sends everything in plaintext: login, commands, and file bodies. Anyone on the same L2 segment (or any intermediate hop) can passively capture:

- **Credentials** — reused across services (as here: FTP password unlocked the backup zip). Credential reuse turns one leak into many.
- **File contents** — backups, configs, key material, customer data.
- **Operational metadata** — who talks to which server, when, how often.

Modern equivalents that still bite:

- **HTTP (not HTTPS)** for internal APIs, admin panels, artifact downloads.
- **Telnet, rlogin, rsh** — still present on legacy gear.
- **SNMP v1/v2c** — community strings in the clear.
- **LDAP simple bind** without TLS.
- **SMTP/POP3/IMAP** without STARTTLS.
- **Database protocols** without TLS (MySQL, Postgres) exposed to a shared VLAN.

On a shared VLAN (cloud, colo, office), one compromised host with a NIC in promiscuous mode can capture all of the above with zero exploitation — just passive listening.

How It Happens in a Real Application

1. **Legacy service never migrated.** FTP has been running since the 2000s because "it works" and the backup script depends on it.
2. **Credentials stuffed into scripts.** `employee13:Harbor_Fal12` lives in a cron job or CI variable. Same password reused to encrypt the archive "for safety."
3. **Flat network.** The FTP server and clients sit on the same VLAN as workstations, so any compromised laptop can sniff.
4. **No monitoring for cleartext protocols.** No alerting on port 21, no policy banning FTP, no TLS enforcement.
5. **Audit finds it late.** A capture reviewed months later shows what already happened.

Fixes (Defensive)

- **Replace FTP with SFTP or FTPS** (explicit TLS). Disable plaintext FTP.
- **Never reuse credentials.** The FTP password should not double as the archive password, and neither should be shared across systems.
- **Segment networks.** Put file-transfer services on their own VLAN, isolated from workstations.
- **Encrypt at rest AND in transit.** If backups must move, use TLS + encrypted archives with separate key material.
- **Enforce TLS everywhere:** HTTPS-only with HSTS, STARTTLS for mail, LDAPS for directory, TLS for DB connections.
- **Detection:** alert on cleartext auth (port 21 login, SNMP v1/v2c, HTTP Basic) crossing the wire. NetFlow/Zeek can flag these.
- **Rotate on leak.** If a capture reveals creds, treat them as compromised — rotate, revoke, and audit for reuse.

Key Takeaways

- Cleartext protocols leak everything: credentials, file content, metadata.
- Reused passwords magnify the blast radius of a single capture.

- Packet captures are a first-class forensic artifact: assume anything unencrypted on the wire is already public.
- Real networks are noisy — filter by protocol, host, and stream; ignore the decoys.

Tools Used

tshark, xxd, grep, unzip, file

Commands That Worked (quick reference)

```
# Protocol overview
tshark -r "$PCAP" -q -z io,phs
tshark -r "$PCAP" -q -z conv,tcp
tshark -r "$PCAP" -q -z endpoints,ip

# FTP creds + commands
tshark -r "$PCAP" -Y "ftp || ftp-data" -T fields \
  -e ip.src -e ip.dst -e ftp.request.command -e ftp.request.arg

# Find ftp-data stream
tshark -r "$PCAP" -Y "ftp-data" -T fields -e tcp.stream

# Carve the transferred file
tshark -r "$PCAP" -q -z follow,tcp,raw,7 \
  | grep -oE '[0-9a-f]{40,}' \
  | xxd -r -p > backup.zip

# Unlock with the leaked password
unzip -P Harbor_Fal12 backup.zip && cat notice.txt
```

Commands That Did NOT Work (and why)

```
# Zsh parse error - angle brackets are redirection
tshark ... -z follow,tcp,raw,<7>
# Fix: bare number 7

# Produced empty output - awk pattern too strict
... | awk 'NF==2 && $1 ~ /^[0-9a-f]+$/ {print $1}' | xxd -r -p
# Fix: grep -oE '[0-9a-f]{40,}'

# tshark field error
... -e http.content_disposition
# Fix: removed; use http.file_data / http.content_type
```